



Hochschule für Angewandte Wissenschaften Hamburg
Hamburg University of Applied Sciences

Informatik 1

BA Medieninformatik

Prof. Dr.-Ing. Sabine Schumann

Fakultät Informatik und Digitale Gesellschaft, Fachgruppe Medieninformatik

Zahlensysteme

There are only 10 types of people in the world:
those who understand binary, and those who don't.

Mathematical joke from Wikipedia

auch lesenswert: <https://blog.codinghorror.com/separating-programming-sheep-from-non-programming-goats/>

Was erwartet Sie in diesem Kapitel?

- historische Zahlensysteme
- Positionssystem
 - Zahlensysteme in der Informatik
 - Addieren im Positionssystem
 - ganze Zahlen
 - gebrochene Zahlen
 - Konvertieren zwischen Positionssystemen (+Rundungsfehler)
- rechnerinterne Darstellung
 - Bits und Bytes
 - Vorzeichen
 - Festkommazahlen, Gleitkommazahlen
 - Character Codierung

Das römische Zahlensystem (1)

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Es stehen folgende Zeichen zur Verfügung

$$II = 1+1 = 2$$

$$CC = 100+100 = 200$$

$$XXX = 10+10+10 = 30$$

$$MMM = 1000+1000+1000 = 3000$$

Das römische Zahlensystem (2)

Regeln:

- I, X, C dürfen nicht mehr als 3mal hintereinander stehen.
 - Steht die kleinere Einheit rechts neben einer größeren, wird die kleinere zur größeren addiert.
 - Steht die kleinere Einheit links neben einer größeren, wird die kleinere von der größeren abgezogen.
 - Es dürfen nicht zwei oder mehrere kleinere von der rechts stehenden größeren Einheit abgezogen werden.
- Bei mehreren möglichen Schreibweisen wird die kürzere gewählt.

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

LXII = ?

XLII = ?

MCMXCIX = ?

MMXXII = ?

weitere Zahlensysteme

Sumerische Zahlzeichen														
	1	2	3	4	5	6	7	8	9	10	60	600	3600	36000
Ägyptische Zahlzeichen														
	1	2	3	4	5	6	7	8	9	10	10 ²	10 ³	10 ⁴	10 ⁵
Altgriechische Zahlzeichen														
	1	2	3	4	5	6	7	8	9	10	50	100	500	1000
Römische Zahlzeichen														
	1	2	3	4	5	6	7	8	9	10	50	100	500	1000
Maya- Zahlzeichen														
	1	2	3	4	5	6	7	8	9	10	0			

[Hoffmann], S. 61, Abbildung 3.2

Positionssysteme (1)

- Ursprung in Indien
- der Weg führte über die Araber nach Europa, weswegen wir es auch als *arabisches System* bezeichnen
- im Gegensatz zu den bisherigen Systemen spielt die **absolute Position** innerhalb einer Ziffernfolge eine entscheidende Rolle für den Zahlenwert
- diese Systeme werden auch *Stellensystem* bzw. *Stellenwertsystem* genannt

Positionssysteme (2)

- Berechnung einer Dezimalzahl z
 z besteht aus den Ziffern $a_{n-1}, \dots, a_0$
dann berechnet sich ihr Wert wie folgt:

$$z = a_0 * 1 + a_1 * 10 + a_2 * 100 + \dots + a_{n-1} * \underbrace{10 \dots 0}_{n-1}$$

$$= \sum_{i=0}^{n-1} a_i * 10^i$$

- das **Dezimalsystem** ist demnach ein spezielles Stellenwertsystem zur **Basis 10**

Positionssysteme (3)

mathematisch gesehen, lässt sich die Basis durch eine beliebige Zahl $b \in \mathbb{N}$ ersetzen

Verallgemeinerte Berechnung:

$$\begin{aligned} z &= \sum_{i=0}^{n-1} a_i * b^i \\ &= a_0 * b^0 + a_1 * b^1 + a_2 * b^2 + \dots + a_{n-1} * b^{n-1} \end{aligned}$$

falls $0 \leq a_i < b$

so ist die Darstellung eindeutig.

Sie erinnern sich ...

... Leibnizsche Binärmaschine & numerische Systeme,
[Historie 17ff](#)

Zahlensysteme in der Informatik

- unser vertrautes **Dezimalsystem**
- **Binärsystem**
- **Oktalsystem**
- **Hexadezimalsystem**

[illegible]

Zahlensysteme in der Informatik – Binärsystem

Binärsystem / Dualsystem

- Basis 2
- Ziffern: 0 und 1
- Computerarchitekturen arbeiten fast ausnahmslos mit nur 2 Zuständen

 **das wichtigste Zahlensystem**

Beispiel:

$$\begin{aligned}n &= (11001)_2 \\&= 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\&= 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 \\&= 16 + 8 + 0 + 0 + 1 \\&= (25)_{10}\end{aligned}$$

$$n = (101010)_2 = ?$$

Zahlensysteme für Informatik – Oktalsystem

Oktalsystem

- Basis 8
 - Ziffern: 0, 1, 2, 3, 4, 5, 6, 7
 - verliert an Bedeutung
 - Konvertierung von Binärzahlen in Oktalzahlen:
 - $8 = 2^3$
-  eine einzige Oktalziffer entspricht also exakt drei Bits in der Binärdarstellung

Beispiel:

$$\begin{aligned} (110111000010)_2 &= \underbrace{110}_6 \underbrace{111}_7 \underbrace{000}_0 \underbrace{010}_2 \\ &= (6702)_8 \end{aligned}$$

Zahlensysteme für Informatik (4)

Hexadezimalsystem

- Basis 16
- Ziffern: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Konvertierung von Binärzahlen in Hexadezimalzahlen:

- $16 = 2^4$

 eine einzige Hexadezimalziffer entspricht also exakt vier Bits in der Binärdarstellung

Beispiel:

$$\begin{aligned} (110111000010)_2 &= \underbrace{1101}_D \underbrace{1100}_C \underbrace{0010}_2 \\ &= (DC2)_{16} \end{aligned}$$

Zahlensysteme in der Informatik

- unser vertrautes **Dezimalsystem**
- **Binärsystem** / Dualsystem
- **Oktalsystem**
- **Hexadezimalsystem**

Dec, Bin, Oct & Hex

dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
bin	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111	10000	10001
oct	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17	20	21
hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11

Ist eine Zahlenbasis aus dem Kontext heraus ersichtlich,
wird auf deren Angabe in der Regel verzichtet.

- alternativ zu $(42)_{10}$ hat sich 42_{10} etabliert
- statt $(42)_{16}$ bzw. 42_{16} gibt es zudem die Präfix-Notation 0x42

Übung

Wandeln Sie die folgenden Zahlen in Dezimalzahlen um:

1. $(C9)_{16} =$
2. $(FEE)_{16} =$
3. $(14)_8 =$
4. $(315)_8 =$
5. $(1100)_2 =$
6. $(11001)_2 =$



20 Minuten

Hex-Speak (1)

Wortursprung:

- Hexadezimal (0123456789ABCDEF)
- englisch: to speak = sprechen
- Zahlen in hexadezimaler Notation, die auch als Wörter der englischen Sprache gelesen werden können
- von Programmierern "erfunden"
- zur Markierung von Daten oder Speicherbereichen mit eindeutigen Identifikator
- neben ABCDEF, *Leetspeak*: Zahlen repräsentieren ähnlich aussehende Buchstaben
 - z. B. Ziffer „5“ den Buchstaben „S“
- Beispiele
 - dezimal 57005 = hexadezimal DEAD
 - DECAFBAD ("koffeinfrei ist schlecht")
 - CODEBA5E
("Codebase" = gesamter Quellcode eines Projektes)

Hex-Speak (2) – Cafe Babe

die ersten 4 Byte **jeder** .class-Datei

0	CA	FE	BA	BE	00	00	00	34	00	1D	0A	00	06	00	0F	09	00	E p ° %		4					
11	10	00	11	08	00	12	0A	00	13	00	14	07	00	15	07	00	16								
22	01	00	06	3C	69	6E	69	74	3E	01	00	03	28	29	56	01	00		< i n i t >		() V				
33	04	43	6F	64	65	01	00	0F	4C	69	6E	65	4E	75	6D	62	65		C o d e		L i n e N u m b e				
44	72	54	61	62	6C	65	01	00	04	6D	61	69	6E	01	00	16	28	r T a b l e		m a i n		(			
55	5B	4C	6A	61	76	61	2F	6C	61	6E	67	2F	53	74	72	69	6E	[L j a v a / l a n g / S t r i n							
66	67	3B	29	56	01	00	0A	53	6F	75	72	63	65	46	69	6C	65	g ;) V		S o u r c e F i l e					
77	01	00	0F	48	65	6C	6C	6F	57	6F	72	6C	64	2E	6A	61	76		H e l l o W o r l d . j a v						
88	61	0C	00	07	00	08	07	00	17	0C	00	18	00	19	01	00	0B	a □							

```
HelloWorld.class
```

Positionssysteme – Arithmetik

Sie können „wie gewohnt“ rechnen.

Lediglich der Übertrag muss entsprechend der zugrundeliegenden Basis gebildet werden.

Dezimal:

$$\begin{array}{r} 7 \\ + 3 \\ \hline = 10 \end{array}$$

Binär:

$$\begin{array}{r} 0111 \\ + 0011 \\ \hline = 1010 \end{array}$$

Oktal:

$$\begin{array}{r} 7 \\ + 3 \\ \hline = 12 \end{array}$$

Hex:

$$\begin{array}{r} 7 \\ + 3 \\ \hline = A \end{array}$$

Positionssysteme – Arithmetik – binär

binäre Addition noch einmal etwas näher beleuchtet:

0 + 0	= 0
0 + 1	= 1
1 + 0	= 1
1 + 1	= 0 Übertrag 1
1 + 1 + 1 (vom Übertrag)	= 1 Übertrag 1

Beispiel:

0101101	45
0110110	54
1111	Übertrag
1100011	99

Positionssysteme – Arithmetik – Übung

Berechnen Sie die dezimale Addition entsprechend in den anderen Zahlensystemen!



20 Minuten

Dezimal:

$$\begin{array}{r} 2712 \\ + 3749 \\ \hline = 6461 \end{array}$$

Binär:

$$\begin{array}{r} 0101010011000 \\ + 0111010100101 \\ \hline = \end{array}$$

Oktal:

$$\begin{array}{r} 5230 \\ + 7245 \\ \hline = \end{array}$$

Hex:

$$\begin{array}{r} A98 \\ + EA5 \\ \hline = \end{array}$$

Konvertieren zwischen Positionssystemen (1)

bisher:

- ✓ binär in dezimal
- ✓ oktal in dezimal
- ✓ hexadezimal in dezimal
- ✓ binär in oktal
- ✓ binär in hexadezimal

jetzt:

- Umrechnen von jeglichem Positionssystem in ein anderes
- Umrechnen beliebiger Zahlen, also auch mit Nachkommastellen

Konvertieren zwischen Positionssystemen (2)

... bspw. dezimal in binär

$(105)_{10}$ zu $(?)_2$

$105 : 2 = 52$ Rest 1

$52 : 2 = 26$ Rest 0

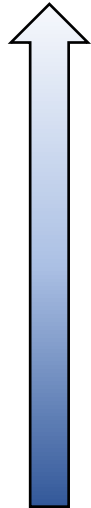
$26 : 2 = 13$ Rest 0

$13 : 2 = 6$ Rest 1

$6 : 2 = 3$ Rest 0

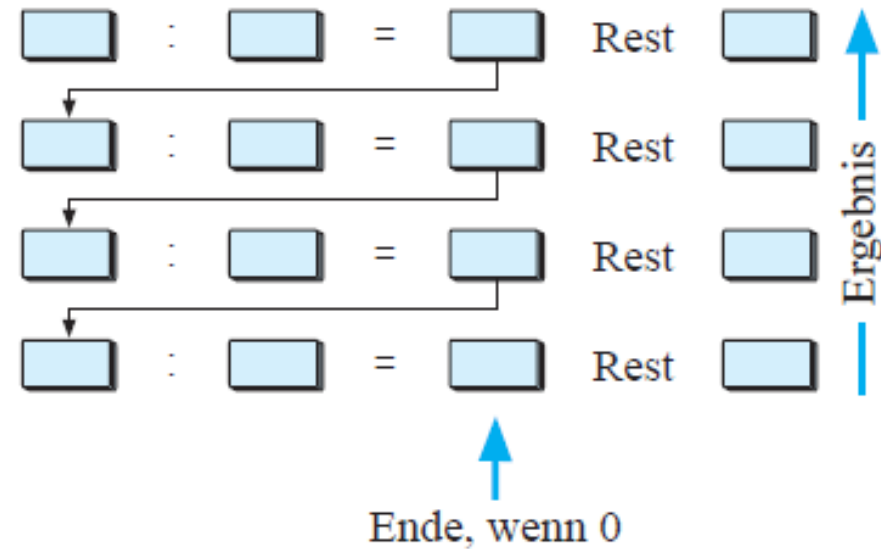
$3 : 2 = 1$ Rest 1

$1 : 2 = 0$ Rest 1



 **1101001**

$(105)_{10} = (1101001)_2$



Bildquelle: [Hoffmann], S. 65, Ausschnitt aus Abbildung 3.3



Zahlen mit Nachkommastellen

unsere bekannte Summenformel:

$$z = \sum_{i=0}^{n-1} a_i * b^i = a_0 * b^0 + a_1 * b^1 + a_2 * b^2 + \dots + a_{n-1} * b^{n-1}.$$

falls $0 \leq a_i < b$

betrachtet bisher nur die n Stellen VOR dem Komma.

$$z = \sum_{i=-m}^{n-1} a_i * b^i$$

berücksichtigt auch die m Nachkommastellen.

Beispiele:

$$(234)_{10} = 2 * 10^2 + 3 * 10^1 + 4 * 10^0$$

$$(234,0)_{10} = 2 * 10^2 + 3 * 10^1 + 4 * 10^0$$

$$(234,702)_{10} = 2 * 10^2 + 3 * 10^1 + 4 * 10^0 + 7 * 10^{-1} + 0 * 10^{-2} + 2 * 10^{-3}$$

$$(0,702)_{10} = 0 * 10^0 + 7 * 10^{-1} + 0 * 10^{-2} + 2 * 10^{-3}$$

Konvertieren zwischen Positionssystemen (3)

... bspw. dezimal in binär

$(0,34375)_{10}$ zu $(?)_2$

$0,34375 * 2 = 0,6875$ Überlauf **0**

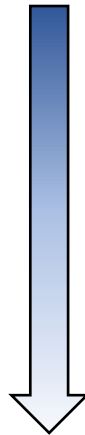
$0,6875 * 2 = 1,375$ Überlauf **1**

$0,375 * 2 = 0,75$ Überlauf **0**

$0,75 * 2 = 1,5$ Überlauf **1**

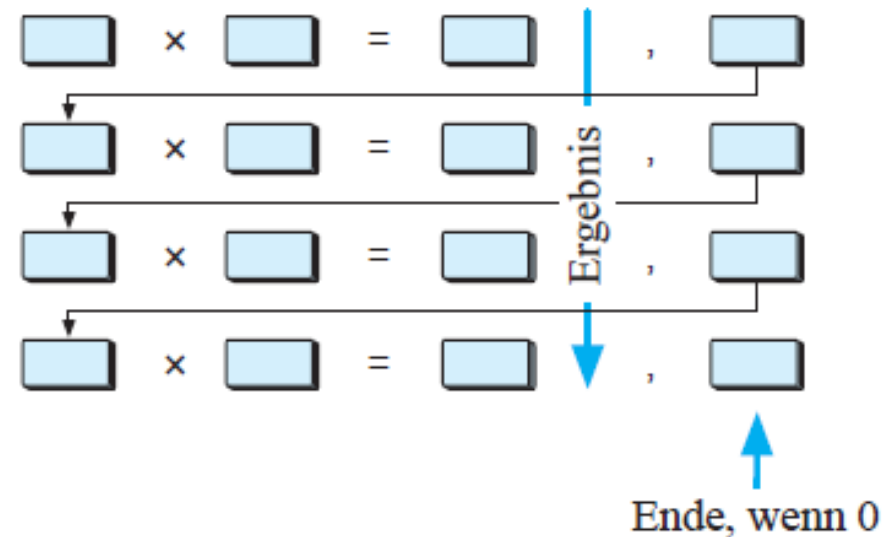
$0,5 * 2 = 1,0$ Überlauf **1**

$0,0 * 2 = 0,0$ Überlauf 0



 **01011**

$(0,34375)_{10} = (0,01011)_2$



Bildquelle: [Hoffmann], S. 65, Ausschnitt aus Abbildung 3.4

Best Practice

- hexadezimal $\rightarrow$ oktal / oktal $\rightarrow$ hexadezimal
hexadezimal $\rightarrow$ binär $\rightarrow$ oktal / oktal $\rightarrow$ binär $\rightarrow$ hexadezimal
- generell bietet sich das Binärsystem als Zwischenformat für alle Positionssysteme, deren Basis auf 2^n beruht, an

ABER:

Beim Konvertieren zwischen Zahlensystemen sollte
IMMER
mit Vorsicht agiert werden, denn ...



... Rundungsfehler und ihre Auswirkungen (1)

Aufgabe: $(0,1)_{10} = (?)_2$

- Zwischenergebnisse wiederholen sich periodisch
- Algorithmus terminiert nicht

$$0,1 * 2 = 0,2 \text{ Übertrag } 0$$

$$0,2 * 2 = 0,4 \text{ Übertrag } 0$$

$$0,4 * 2 = 0,8 \text{ Übertrag } 0$$

$$0,8 * 2 = 1,6 \text{ Übertrag } 1$$

$$0,6 * 2 = 1,2 \text{ Übertrag } 1$$

$$0,2 * 2 = 0,4 \text{ Übertrag } 0$$

$$0,4 * 2 = 0,8 \text{ Übertrag } 0$$

$$0,8 * 2 = 1,6 \text{ Übertrag } 1$$

$$0,6 * 2 = 1,2 \text{ Übertrag } 1$$

$$0,2 * 2 = \dots$$

Konvertieren zwischen Zahlensystemen ist eine klassische aber in der Praxis wenig beachtete Quelle von durchaus **fatalen Fehlern.**



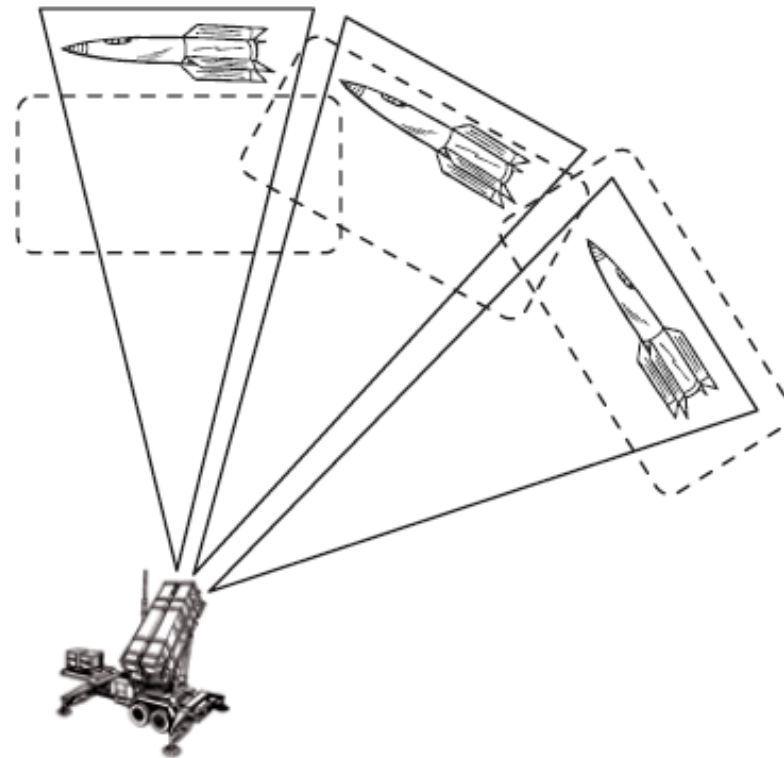
... Rundungsfehler und ihre Auswirkungen (2)

- Patriot – bodengestütztes Kurzstrecken-Flugabwehrsystem
- im 1. Golfkrieg ab 01/1991 eingesetzt, um irakische „Scud“ und „Al-Hussein“-Raketen abzufangen
- 25.02.1991:
 - eine Scud trifft eine Kaserne in Dhahran
 - 28 Tote, über 90 Schwerverletzte
 - Patriot hatte die Rakete erkannt
 - feuerte jedoch keine Abwehrrakete ab



... Rundungsfehler und ihre Auswirkungen (3)

- aufgrund der Potenzierung des Rundungsfehlers war ab einer Betriebsdauer von 20 Stunden das Zielgebiet soweit verschoben, dass sich jedes Zielobjekt außerhalb des Areals befand
- Patriot war zu diesem Zeitpunkt bereits über 100 Stunden im Einsatz (was eine Zielverschiebung von ca. 500m entspricht)



[Hoffmann], S. 66, Wikipedia

Rechnerinterne Zahlendarstellung (1)

Bits und Bytes

bilden die Grundlage zur Datenspeicherung

Bit = binary digit

- kleinste elektronische Speichereinheit
- an und aus / 0 und 1
- 2 Möglichkeiten

Byte

- kleinste Datenmenge
- Bits werden zu einer Speichereinheit zusammengefasst
- **8 Bits** ergeben ein Byte
- Bytes können in höhere Einheiten umgerechnet werden
- 256 Möglichkeiten

Rechnerinterne Zahlendarstellung (2)

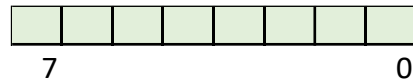
- Anzahl der Bytes, die zur Repräsentation einer einzelnen Zahl (intern) verwendet wird, definiert den darstellbaren Zahlenbereich (und entspricht i.a. einer Zweierpotenz 2^n)
- betrachten wir (erstmal) die positiven (ganzen) Zahlen:

Byte	Bit	Wertebereich
1	8	$[0; 2^8-1] = [0; 255]$
2	16	$[0; 2^{16}-1] = [0; 65.535]$
4	32	$[0; 2^{32}-1] = [0; 4.294.967.295]$
8	64	$[0; 2^{64}-1] = [0; \sim 1,84 \cdot 10^{19}]$

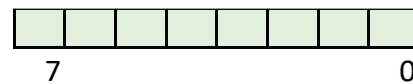
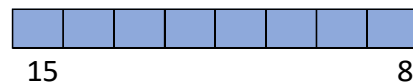
Rechnerinterne Zahlendarstellung (3)

Speicherordnung

- Little- Endian:
Byte mit der niedrigsten Wertigkeit zuerst
bereits mit Intel 4004 eingeführt



- Big-Endian:
Byte mit der höchsten Wertigkeit zuerst
scheint „natürlicher“



Kriterien für Zahlenformate (1)

- Arithmetik
 - arithmetische Operationen gehören zu den Kernaktivitäten eines jeden Rechnersystems
 - die Eignung einer Zahlendarstellung kann also daran gemessen werden, wieviel Aufwand elementare Rechenoperationen verursachen
- Eineindeutigkeit
 - ein Zahlenformat ist eineindeutig, wenn jede Zahl **genau einem Bitmuster** und **umgekehrt** entspricht
 - damit lässt sich der Test auf Gleichheit effizient implementieren
 - es wird kein Bit „verschenkt“
 - nicht eindeutige Zahlensysteme werden auch als *redundante Zahlensysteme* bezeichnet

Kriterien für Zahlenformate (2)

- Symmetrie
 - ein Zahlenformat ist symmetrisch, wenn zu jeder Zahl z auch die negative Zahl $-z$ dargestellt werden kann
 - diese Eigenschaft ist erstrebenswert, da dann die Negationsoperation innerhalb der Menge der darstellbaren Zahlen abgeschlossen ist
 - Unsymmetrie kann zu unvorhersehbaren Wertüberläufen führen
 - der einfache Vorzeichenwechsel kann bei Unsymmetrie zu komplett falschen Resultaten führen

Vorzeichen ...

Dezimal:

$$\begin{array}{r} 101 \\ - 122 \\ \hline = -21 \end{array}$$

negative Zahlen werden i.a. durch ihren Betrag mit einem vorangestellten Minuszeichen notiert

Vorzeichen ... das ausgezeichnete Vorzeichenbit

Dezimal:

$$\begin{array}{r} 101 \\ - 122 \\ \hline = -21 \end{array}$$

Binär:

$$\begin{array}{r} 01100101 \\ - 01111010 \\ \hline = ? \end{array}$$

- die einfachste Möglichkeit:
Darstellung des Vorzeichens durch ein ausgezeichnetes festes Bit:
bspw. das **Höchstwertige** mit
 - positiv: 0 und negativ: 1.
 - leichtes Invertieren von Zahlen (Kippen des Vorzeichens)
 - symmetrisch

Symmetrie bei beispielhafter 8-Bit-Darstellung

Binärwert	Interpretation als vorzeichenbehaftete Dezimalzahl
01111111	127
01111110	126
...	...
00000010	2
00000001	1
00000000	0
10000000	-0
10000001	-1
10000010	-2
...	...
11111110	-126
11111111	-127

Vorzeichen ... das ausgezeichnete Vorzeichenbit

Dezimal:

$$\begin{array}{r} 101 \\ - 122 \\ \hline = -21 \end{array}$$

Binär:

$$\begin{array}{r} 01100101 \\ - 01111010 \\ \hline = ? \end{array}$$



nicht eindeutig:

+0 = 0 00..00

- 0 = 1 00..00

- Fallunterscheidung beim Rechnen notwendig! (aufwendig!!!)
- Fehler beim Rechnen ...

Rechenbeispiele:

Dezimal:

$$\begin{array}{r} 5 \\ + -6 \\ \hline = -1 \end{array}$$

Binär:

$$\begin{array}{r} 0101 \quad (5) \\ + 1110 \quad (-6) \\ \hline = 10011 \quad (-3) \end{array}$$

Dezimal:

$$\begin{array}{r} 5 \\ + -3 \\ \hline = 2 \end{array}$$

Binär:

$$\begin{array}{r} 0101 \quad (5) \\ + 1011 \quad (-3) \\ \hline = 10000 \quad (-0) \end{array}$$

Vorzeichen ... Einerkomplement

- beim Einerkomplement gibt es kein besonderes Vorzeichenbit
- stattdessen wird das Bitmuster einer Zahl z vollständig invertiert
- der abgedeckte Zahlenbereich ist wiederum symmetrisch
- das Problem mit der Doppeldarstellung der 0 bleibt
- Addition erfolgt beim Einerkomplement in 3 Schritten:
 1. Ausführen der gewöhnlichen Binäraddition
 2. Aufaddieren des Übertrags (**Übertragsadditionsregel**)
 3. Streichen verbleibender Überträge

Symmetrie bei beispielhafter 8-Bit-Darstellung

Binärwert	Interpretation als vorzeichenbehaftete Dezimalzahl
01111111	127
01111110	126
...	...
00000010	2
00000001	1
00000000	0
11111111	-0
11111110	-1
11111101	-2
...	...
10000001	-126
10000000	-127

Rechenbeispiele für 4-Bit:

Dezimal:

$$\begin{array}{r} 5 \\ + -6 \\ \hline = -1 \end{array}$$

Binär:

$$\begin{array}{r} 0101 \quad (5) \\ + 1001 \quad (-6) \\ \hline = 1110 \quad (-1) \end{array}$$

ausführlich:

$$\begin{array}{r} 0101 \quad (5) \\ + 1001 \quad (-6) \\ \hline = 0|1110 \quad (-1) \\ + 0 \\ \hline = \cancel{0}|1110 \quad (-1) \end{array}$$

Dezimal:

$$\begin{array}{r} 5 \\ + -3 \\ \hline = 2 \end{array}$$

Binär:

$$\begin{array}{r} 0101 \quad (5) \\ + 1100 \quad (-3) \\ \hline = 1|0001 \quad (-14) \\ + 1 \quad (1) \\ \hline = \cancel{1}|0010 \quad (2) \end{array}$$

Vorzeichen ... Zweierkomplement

- behebt das Problem mit der Doppelnull
(weswegen die Übertragsadditionsregel angewendet werden muss)
- Vorgehen zum Negieren einer Zahl:
 1. Einerkomplement durch Kippen aller Bits gebildet
 2. anschließend eine 1 dazu addieren.
- darstellbarer Zahlenbereich: $-2^{n-1}, \dots, 0, \dots, 2^{n-1}-1$
- Symmetrieeigenschaft geht verloren
der negative Bereich ist nun um 1 größer als der positive
- Addition erfolgt beim Zweierkomplement in 2 Schritten:
 1. Ausführen der gewöhnlichen Binäraddition
 2. Streichen verbleibender Überträge
- Zweierkomplement ist die am häufigsten eingesetzte Darstellung für vorzeichenbehaftete ganze Zahlen

Zweierkomplement bei beispielhafter 8-Bit-Darstellung

Binärwert	Interpretation als vorzeichenbehaftete Dezimalzahl
01111111	127
01111110	126
...	...
00000010	2
00000001	1
00000000	0
11111111	-1
11111110	-2
...	...
10000010	-126
10000001	-127
10000000	-128

Rechenbeispiele für 4-Bit:

Dezimal:

$$\begin{array}{r} 5 \\ + -6 \\ \hline = -1 \end{array}$$

Binär:

$$\begin{array}{r} 0101 \quad (5) \\ + 1010 \quad (-6) \\ \hline = 1111 \quad (-1) \end{array}$$

Dezimal:

$$\begin{array}{r} 5 \\ + -3 \\ \hline = 2 \end{array}$$

Binär:

$$\begin{array}{r} 0101 \quad (5) \\ + 1101 \quad (-3) \\ \hline = \cancel{1}0010 \quad (2) \end{array}$$

Zweierkomplement und die Asymmetrie ...

▶ Die Asymmetrie kann in der Praxis fatale Folgen haben, wenn **der kleinste darstellbare negative Wert invertiert** wird

Beispiel:

bei Breite 4 Bit:

kleinste darstellbare Zahl: $-8 = 1000$

wir wollen $-(-8)$ berechnen

1. Invertieren: $1000 \rightarrow 0111$
2. 1 dazu addieren $0111+1 = 1000$ also wieder -8

Übung – Zweierkomplement

Nehmen wir an, wir haben einen Datentyp für Ganzzahlen (positive und negative) mit der Größe 4 Bit im Zweierkomplement erschaffen.

Welche (Dezimal-)Zahlen können wir damit darstellen?

$$-8 \rightarrow 7$$



5 Minuten

Darstellung rationaler Zahlen - Festkommazahlen

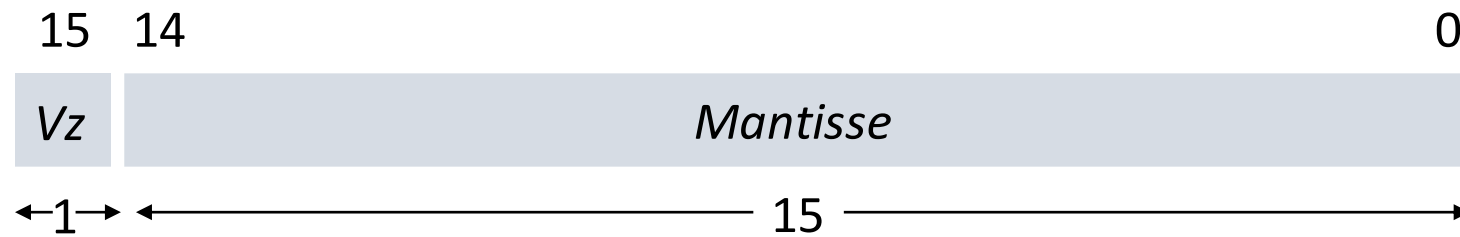
Eine **Festkommazahl** ist eine Zahl, die aus einer festen Anzahl von Ziffern besteht. Die Position des Kommas ist dabei fest vorgegeben, daher der Name. Der Grundgedanke dahinter ist die informationstechnische Repräsentation eines Ausschnitts der rationalen Zahlen.

- das erste Bit bestimmt, ob es sich um eine positive oder negative Zahl handelt
- in den restlichen Bits werden die Nachkommastellen abgebildet (*Mantisse*)
- die Festkomma-Darstellung erlaubt eine Repräsentation von Zahlen aus dem offenen Intervall $]-1; 1[$
- aufgrund dieses Nachteils wird die Festkomma-Darstellung nur noch in Spezialrechnern für spezielle Anwendungen verwendet (Signalverarbeitung)
- heute übliche Rechner benutzen die Gleitkomma-Darstellung

Festkommazahlen-Format

Beispiel:

Bitbreite: 16 Bit



Darstellung rationaler Zahlen - Gleitkommazahlen

- erweitert die Festkomma-Darstellung um einen Exponenten
- englisch: *floating point number*, deswegen auch *Fließkommazahl* gebräuchlich
- Durchgesetzt hat sich die IEEE-754-Norm vom Institute of Electrical and Electronics Engineers
IEEE 754-1985
IEEE 754-2008: binary16, binary32, binary64, binary128
IEEE 754-2019
(andere Formate Zuse Z1, IBM S/390 single, ...)
- IEEE 754-2008 wird hier vorgestellt

Gleitkommazahlen-Format

18,625 in Binär?

$$(18)_{10} = (10010)_2$$

$$(0,625)_{10} = (101)_2$$

$$(18,625)_{10} = (10010,101)_2$$

$$\begin{aligned} 10010,101 &= 10010,101 * 2^0 \\ &= 1001,0101 * 2^1 \\ &= 100,10101 * 2^2 \\ &= 10,010101 * 2^3 \\ &= 1,0010101 * 2^4 \end{aligned}$$



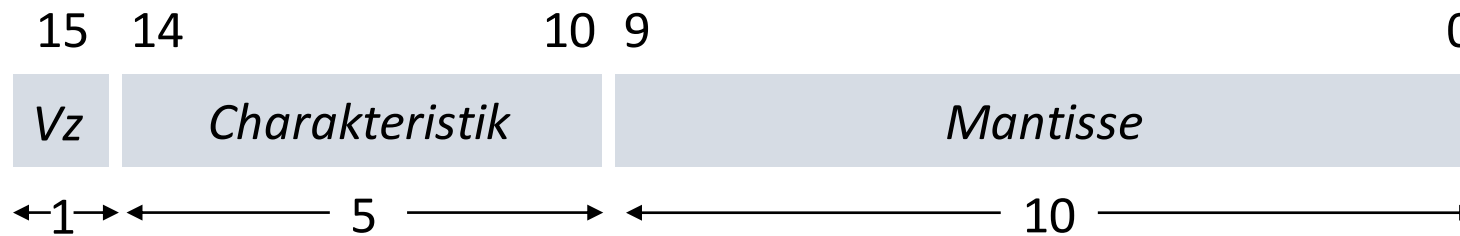
Normalisierung,
bis 1,... erreicht ist

wir müssen also das Vorzeichen, 1,**0010101** und 2^4
in der internen Zahlendarstellung unterbringen

Gleitkommazahlen-Format

uns stehen z. B. **16 Bit** zum Speichern zur Verfügung
IEEE: binary16 (half-precision, s10e5)

- 1 Bit für das Vorzeichen
- 10 Bit für die Ziffernfolge der Zahl (Mantisse)
- 5 Bit für die Charakteristik (Exponent + Bias)



In unserem Beispiel $1,0010101 \cdot 2^4$

Vorzeichen: 0

Mantisse: 0010101000

Charakteristik = 4 + Bias

Bias

Charakteristik = Exponent + Bias

- in unserem Beispiel haben wir bei binary16 **5** Bits zur Verfügung, um die Charakteristik abzuspeichern
- wir wollen sowohl negative als auch positive Exponenten darstellen, in etwa die gleiche Anzahl
- d.h. wir wählen den Exponenten 0 in etwa der Mitte unseres Zahlenbereiches
- 5 Bit = 32 darstellbare ganze Zahlen
- Bias eine Stelle unterhalb der Hälfte: damit 15

zurück zum Beispiel

In unserem Beispiel $1,0010101 \cdot 2^4$

Vorzeichen: 0

Mantisse: 0010101000

Exponent: 4

Charakteristik = Exponent + Bias = $4 + 15 = 19$

$(19)_{10} = (10011)_2$

damit 18,625 auf 16 Bit im Format s10e5:

0 10011 0010101000

IEEE – Formate

Name	Bit	Genauigkeit	Werte des Exponenten <i>e</i>	Bias	Aufteilung der Bits <i>Vz+Charakteristik+Mantisse</i>
binary16	16	half	$-14 \leq e \leq 15$	15	1+5+10
binary32	32	single	$-126 \leq e \leq 127$	127	1+8+23
binary64	64	double	$-1022 \leq e \leq 1023$	1023	1+11+52
binary128	128	quad	$-16382 \leq e \leq 16383$	16383	1+15+112

Ungenauigkeiten bei Gleitkommazahlen

- Gleitkommazahlen, die im Dezimalsystem exakt dargestellt werden können, können im Binärsystem nicht immer ganz genau abgebildet werden
- dadurch kleine Ungenauigkeiten, die sich jedoch aufsummieren können
- grundsätzlich sollten float- oder double-Werte niemals auf Gleichheit geprüft werden

Übung

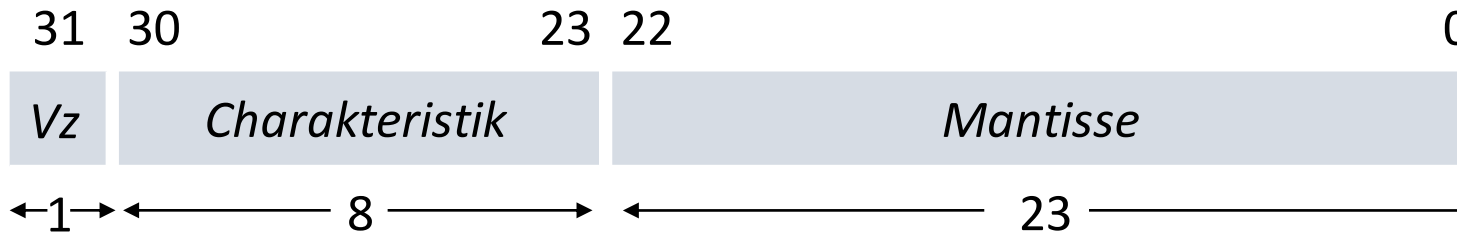
$(-18,4)_{10}$ in **binary32** (single, s23e8) darstellen

10000010

$$127 + 4 = 131 = 128 + 3$$



15 Minuten



$$Vz = 1$$

$$(0,4)_{10} = 011001100110_2 = 1,001001100110 \dots \cdot 2^4$$

$$(18)_{10} = 10010_2$$

$$(18,4)_{10} = 10010,011001100110$$

Charakter Codierung – ASCII

- **American Standard for Coded Information Interchange (ASCII)**
- Standard seit 1963
- jedes Zeichen wird in einem Byte gespeichert, jedoch wird das erste Bit im Standard-ASCII nicht benutzt, so dass $2^7=128$ Zeichen dargestellt werden können
- das achte Bit soll die Überprüfung der Daten auf Fehler ermöglichen (Paritätsbit), dient allerdings nur dazu, dass der Empfänger der Bitfolge darauf hingewiesen wird, dass ein Fehler vorliegt

aus diesem Grund gibt es auch ASCII-Codes mit 256 Zeichen
(das 8. Bit wird auch für die Kodierung verwendet)

ASCII (1)

Zeichenkodierung umfasst 128 Zeichen, davon 95 druckbare und 33 nicht druckbare

- diese werden dezimal von 0 bis 127 nummeriert
- 65- 90 Großbuchstaben des lateinischen Alphabets
- 97-122 Kleinbuchstaben des lateinischen Alphabets
- 48- 57 arabische Ziffern
- 32- 47, 58-64, 91-96, 123-126 Sonderzeichen
- 0- 31, 127 Steuerzeichen

ASCII-Tabellen-Ausschnitt darstellbarer Zeichen

Zeichen	Dezimal	Binär	Oktal	Hex
/	47	0010 1111	057	2F
0	48	0011 0000	060	30
1	49	0011 0001	061	31
2	50	0011 0010	062	32
3	51	0011 0011	063	33
=	61	0011 1101	075	3D
A	65	0100 0001	101	41
B	66	0100 0010	102	42
C	67	0100 0011	103	43
D	68	0100 0100	104	44
a	97	0110 0001	141	61
b	98	0110 0010	142	62
c	99	0110 0011	143	63
d	100	0110 0100	144	64

ASCII (2)

- Speichern von Texten:
 - Bytes, die jeweils ein Zeichen kodieren werden hintereinander gespeichert, es entsteht eine Zeichenkette
 - um das Ende eine Zeichenkette zu markieren, werden (in Programmiersprachen) unterschiedliche Verfahren angewendet
- '1' versus 1 (in C/C++, Java)

Ziffer als ASCII	Dezimal	Binär
'1'	49	00110001
'8'	56	00111000
Ziffer numerisch	Dezimal	Binär
1	1	00000001
8	8	00001000

Charakter Codierung - Unicode

- ASCII ist mit 128 (bzw. 256) Zeichen sehr begrenzt
- mit Unicode können alle Zeichen bekannter Schrift- und Zeichensysteme abgespeichert werden
- Unicode Konsortium
- <http://www.unicode.org/>
- Darstellung hexadezimal mit vorangestelltem *U+*
- Zeichen lassen sich dynamisch kombinieren
 - es gibt das Zeichen *ä* sowie
 - die Kombination von *a* mit dem Element für Doppelpunkt über einem Zeichen (Trema) " (U+0308)
voila: *ä, ë, ö, x ...*
- in Zeichenbereiche eingeteilt

weitere Codes für Zahlen und Zeichen: Tetraden

- Tetraden-Codes sind Binärcodes, die eine Zahl ziffernweise kodieren
- für die Kodierung der Dezimalziffern 0 bis 9 werden demnach nur zehn Bitmuster benötigt
- es gibt also in einem Tetraden-Code sechs *Pseudo-Tetraden*, die nicht verwendet werden

Tetraden-Codes:

- BCD
- Gray-Code
- erweiterter Gray-Code
- Stibitz-Code
- Aiken-Code
- ...

Pseudotetraden

- Pseudotetraden treten bei binär codierten Dezimalzahlen auf
- es handelt sich hierbei um Codes, die nicht benutzt werden (Don't Care)
- für 10 Ziffern werden 4 Bit benötigt –
4 Bit erzeugen aber 16 Codierungen
- d.h. 6 Codierungen werden nicht benutzt, die sogenannten Pseudotetraden

Tetraden: BCD

- Binary Coded Decimales (BCD)
z.Bsp. zur Ansteuerung von LCDs

Dezimalzahl	Binärzahl	binäre BCD-Darstellung
294	100100110	0010.1001.0100 2 9 4
16289	11111110100001	0001.0110.0010.1000.1001 1 6 2 8 9

Bit-Nr.	4	3	2	1
Wertigkeit	2^3 8	2^2 4	2^1 2	2^0 1
Dezimalziffern	0	1	2	3
4	5	6	7	8
9				

Bildquelle: https://de.wikipedia.org/wiki/BCD-Code#/media/Datei:Code_BCD.svg

Tetraden: Gray-Code

- Gray-Code
z.Bsp. für binäre Ausgabe von A/D-Wandlern
2 aufeinanderfolgende Codewörter unterscheiden sich nur um genau 1 Bit

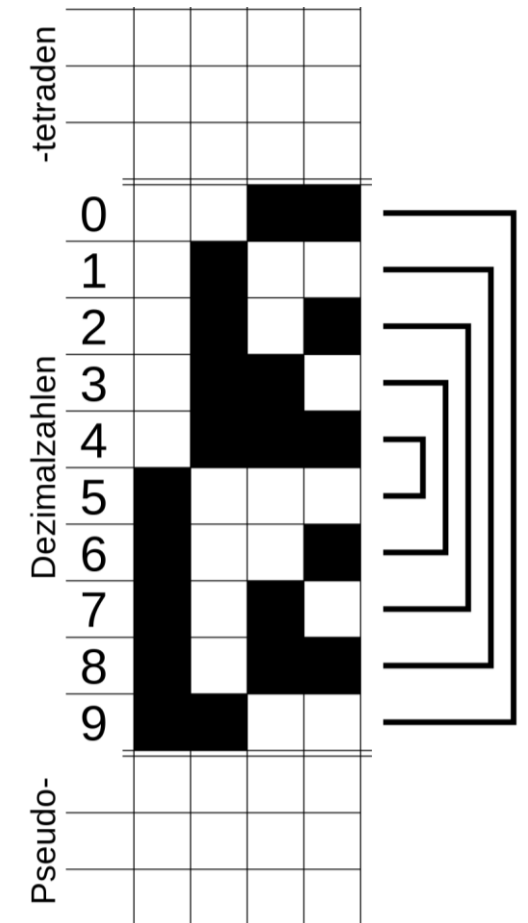
Dezimalzahl binäre Gray-Darstellung

1	0001
2	0011
3	0010
4	0110
5	0111

Tetraden: Stibitz-Code

- Stibitz-Code (Excess-3-Code)
im Vergleich zum BCD sind die Bitmuster um 3 verschoben (d.h. zu jeder BCD-Tetrade 3 (0011) addieren),
der Stibitz-Code ist symmetrisch

Dezimalzahl	Binärzahl	binäre Stibitz-Darstellung
294	100100110	0101.1100.0111 2 9 4
16289	11111110100001	0100.1001.0101.1011.1100 1 6 2 8 9



Bildquelle: https://de.wikipedia.org/wiki/Stibitz-Code#/media/Datei:Exzess3_codetafel_symmetrie.svg

Tetraden: Aiken-Code

- Aiken-Code (2421-Code)
ist ein komplementärer BCD
der Aiken-Code ist symmetrisch

Dezimalzahl	Binärzahl	binäre Aiken-Darstellung
294	100100110	0010.1111.0100 2 9 4
16289	11111110100001	0001.1100.0010.1110.1111 1 6 2 8 9

		Aiken-Code				Bit
		3	2	1	0	
		2	4	2	1	Wertigkeit
Dezimalzahlen	0					
	1					
	2					
	3					
	4					
Pseudotetraden						
Dezimalzahlen	5					
	6					
	7					
	8					
	9					

Bildquelle: https://de.wikipedia.org/wiki/Aiken-Code#/media/Datei:Aiken_codetafel_symmetrie.PNG

Gegenüberstellung der Tetraden-Codes

Dezimal	BCD	Gray	Stibitz	Aiken
0	0000	0000	0011	0000
1	0001	0001	0100	0001
2	0010	0011	0101	0010
3	0011	0010	0110	0011
4	0100	0110	0111	0100
5	0101	0111	1000	1011
6	0110	0101	1001	1100
7	0111	0100	1010	1101
8	1000	1100	1011	1110
9	1001	1101	1100	1111
10	0001 0000	0001 0000	0100 0011	0001 0000
11	0001 0001	0001 0001	0100 0100	0001 0001
12	0001 0010	0001 0011	0100 0101	0001 0010

weitere Codes für Zahlen und Zeichen

- Barcode / Strichcode
keine interne Rechnerverwendung, aber Dekodierung in ASCII- oder BCD-Kodierung



weitere Codes für Zahlen und Zeichen

fehlererkennende Codes

- für die Datenübertragung
- Bitmuster unterscheiden sich in möglichst vielen Bitstellen
- „kippt“ ein Bit entsteht ein nicht verwendetes Codemuster

Fehlererkennende Codes:

- 74210-Code
- Interleaved-2-aus-5-Code
- Walking-Code
- Biquinär-Code
- reflektiver Biquinär-Code
- One-Hot-Code

Maßeinheiten für Bytes (1)

1 Byte		8 Bit
1 Kilobyte (KByte)	2^{10}	1024 Byte
1 Megabyte (MByte)	2^{20}	1024 KByte
1 Gigabyte (GByte)	2^{30}	1024 MByte
1 Terabyte (TByte)	2^{40}	1024 GByte
1 Petabyte (PByte)	2^{50}	1024 TByte
1 Exabyte (EByte)	2^{60}	1024 PByte
1 Zettabyte (ZByte)	2^{70}	1024 EByte
1 Yottabyte (YByte)	2^{80}	1024 ZByte
...		...

Achtung:

KB nicht gleich KByte, Hersteller nehmen als Basis 10

$10^3 \neq 2^{10}$, $1000 \neq 1024$

bei 1 GB zu 1 GByte fehlen demnach 73 MByte

eine 200 GB Festplatte ist demnach nur 186 GByte groß



Maßeinheiten für Bytes (2)

- SI-Einheiten beziehen sich auf Basis 10
- bis 1996 keine speziellen Einheitenvorsätze für Zweierpotenzen
- wenn eine Informatiker*in 1KiloByte sagt, meint er/sie 1024 Bytes
- wenn „irgendwo“ 1KiloByte steht kann man nicht sicher sein, ob 1000 oder 1024 Bytes gemeint sind
- das für die SI-Präfixe zuständige Internationale Büro für Maß und Gewicht (BIPM) rät von dieser nicht normgemäßen Verwendung der SI-Präfixe ausdrücklich ab
- es empfiehlt für die Bezeichnung von Zweierpotenzen die Binärpräfixe gemäß IEC 60027-2. (**K**ibibyte, **M**ebibyte, **G**ibibyte, ...)

Diese empfohlene Bezeichnungsweise ist bis heute nicht sehr weit verbreitet.

Zudem:

Das ganze ist kein Standard, denn SI-Präfixe sind nur für SI-Einheiten standardisiert; und Byte ist keine SI-Einheit.



Lernziele

- ... Positionssysteme verstanden haben
- ... Zahlen eines Positionssystems in ein anderes konvertieren können
- ... Addition von Binärzahlen beherrschen
- ... Tetraden-Kodierung verstanden haben
- ... die Idee von Positionssystemen auf andere Problemstellungen übertragen können
- ... Fehlerhaftigkeit dabei IMMER im Blick haben